

GUIA PRÁTICO PARA ESTUDANTES

Git + GitHub

do zero ao primeiro push

Uma sequência de estudo usando um projeto local real, entendendo o que cada comando faz e como o código sai do computador e chega ao GitHub.



ANTES DE COMEÇAR · VISÃO GERAL

Como o Git funciona?

Imagine o Git como uma **linha do tempo do projeto**. Cada desenvolvedor trabalha em sua própria cópia, registra pontos seguros e depois compartilha esses pontos com a equipe.



Ideia principal: o Git controla versões localmente. O GitHub permite compartilhar essas versões e colaborar pela internet.

ANTES DE COMEÇAR · HISTÓRICO

Commits formam a história do software



Cada círculo representa um **commit**: um ponto identificado no histórico. Isso permite saber **quem alterou, o que mudou e em qual sequência**.

Terminal / Git Bash

```
git log --oneline
```

ANTES DE COMEÇAR · BRANCH

Branch = uma linha paralela de trabalho

Uma branch permite desenvolver uma funcionalidade sem alterar diretamente a versão principal.



main

Representa a linha principal e, normalmente, a versão integrada do projeto.

feature/login

Espaço separado para implementar o login, testar e criar commits sem interromper a main.

ANTES DE COMEÇAR · BRANCH NA PRÁTICA

Crie uma branch para sua tarefa

Terminal / Git Bash

```
# veja onde você está
git branch

# crie a branch e já entre nela
git switch -c feature/login

# confirme a branch atual
git branch
```

O símbolo * indica a branch em que você está trabalhando.

```
* feature/login
main
```

Boa prática didática: uma tarefa ou funcionalidade → uma branch. Exemplos: feature/login, feature/cadastro, fix/validacao-email.

Cada pessoa tem seu próprio repositório local



Git é **distribuído**: Ana e João possuem uma cópia completa do repositório e do histórico. Eles podem criar commits localmente e sincronizar quando necessário.

Ana

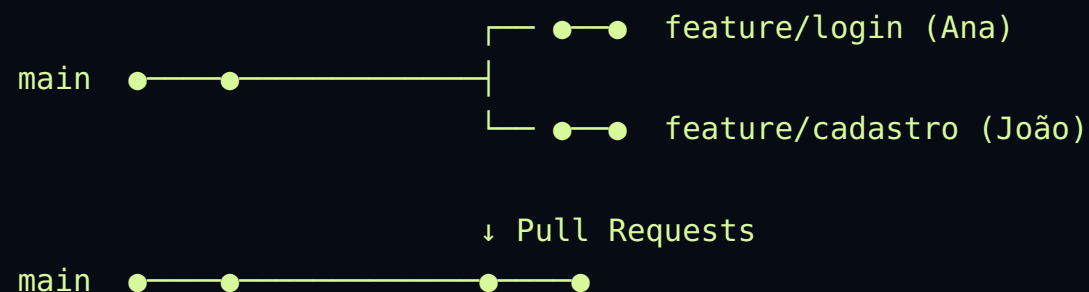
feature/login
trabalha no login.

João

feature/cadastro
trabalha no cadastro.

ANTES DE COMEÇAR · EQUIPE

Exemplo: duas branches ao mesmo tempo



Cada pessoa faz commits na própria branch, envia com `git push` e abre um **Pull Request**. Depois da revisão, as alterações podem ser integradas à `main`.

ANTES DE COMEÇAR · CICLO COLABORATIVO

Fluxo de trabalho distribuído



Terminal / Git Bash

```
git pull
git switch -c feature/minha-tarefa
# editar arquivos
git add .
git commit -m "feat: implementa minha tarefa"
git push -u origin feature/minha-tarefa
```

Esse mapa será retomado mais adiante. Primeiro vamos aprender cada comando separadamente.

Comandos que vamos exercitar

Use esta página como checklist durante a aula. A sequência acompanha o ciclo real de um projeto: configurar, criar, versionar, publicar, atualizar e trabalhar em branch.

1. Preparar e criar

```
git --version
git config --global user.name "Seu Nome"
git config --global user.email "email@exemplo.com"
git config --global --list
mkdir projeto-git
cd projeto-git
git init -b main
```

2. Versionar localmente

```
git status
git add README.md
git add .
git diff
git commit -m "docs: primeiro commit"
git log --oneline
```

3. Conectar e sincronizar

```
git remote add origin URL
git remote -v
git push -u origin main
git pull
git clone URL
```

4. Trabalhar com branch

```
git branch
git switch -c feature/minha-tarefa
git add .
git commit -m "feat: minha tarefa"
git push -u origin feature/minha-tarefa
```

GitHub não é o único servidor Git

O **Git** é o sistema de controle de versão. Já serviços como GitHub, GitLab, Bitbucket e Azure Repos hospedam repositórios remotos e adicionam recursos de colaboração.

GitHub

Hospedagem de repositórios Git, colaboração, Pull Requests e automação. Será a plataforma principal dos nossos exemplos.

GitLab

Plataforma de DevSecOps com repositórios Git e recursos integrados de planejamento, CI/CD, segurança e entrega.

Bitbucket

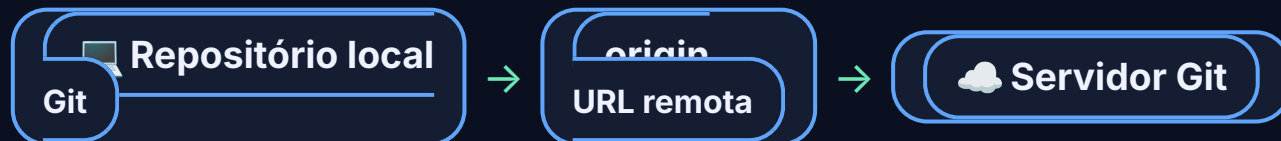
Hospedagem e colaboração Git da Atlassian, com integração forte ao Jira e CI/CD pelo Bitbucket Pipelines.

Azure Repos

Serviço do Azure DevOps para repositórios Git, Pull Requests, revisão de código e políticas de branch.

Importante: os comandos Git continuam praticamente os mesmos. O que muda principalmente é a URL do repositório remoto e os recursos oferecidos pela plataforma.

O mesmo projeto pode apontar para diferentes servidores



```
git remote add origin https://github.com/aluno/projeto.git
```

Exemplo com GitHub

```
git remote add origin https://gitlab.com/aluno/projeto.git
```

Exemplo com GitLab

```
git remote add origin https://bitbucket.org/aluno/projeto
```

Exemplo com Bitbucket

```
git remote add origin https://dev.azure.com/org/projeto/_
```

Exemplo de formato do Azure Repos

Depois de configurar o remoto, o fluxo continua familiar: `git push`, `git pull` e `git fetch`.

01 · CONCEITO

Git não é GitHub

Git

Sistema de controle de versão instalado no computador.
Registra o histórico das alterações do projeto.

GitHub

Plataforma online que hospeda repositórios Git e facilita colaboração, revisão e compartilhamento.

Imagem mental: Git é a máquina fotográfica do histórico; GitHub é o álbum online compartilhado.

02 · PREPARAÇÃO

Instale e confira o Git

Terminal / Git Bash

```
git --version
```

Se aparecer uma versão, o Git está disponível. Depois abra Git Bash, Terminal ou terminal integrado da IDE.

Objetivo: antes de criar repositórios, garantir que o computador reconhece o comando **git**.

03 · IDENTIDADE

Configure nome e e-mail

Terminal / Git Bash

```
git config --global user.name "Seu Nome"  
git config --global user.email "seuemail@exemplo.com"  
  
git config --global --list
```

Esses dados identificam o autor dos commits. **--global** aplica a configuração aos seus repositórios no computador.

04 · PROJETO LOCAL

Crie uma pasta de estudo

Terminal / Git Bash

```
mkdir projeto-git  
cd projeto-git
```

No Windows/Git Bash use `cd projeto-git`. O “a” acima não é necessário — use somente o comando abaixo:

```
cd projeto-git
```

Crie dentro dela um arquivo `README.md` e, se quiser, um `index.html`.

05 · INICIALIZAÇÃO

Transforme a pasta em repositório

Terminal / Git Bash

```
git init -b main
```



O diretório oculto `.git` guarda metadados e histórico. Não edite essa pasta manualmente.

06 · DIAGNÓSTICO

Seu comando mais frequente

Terminal / Git Bash

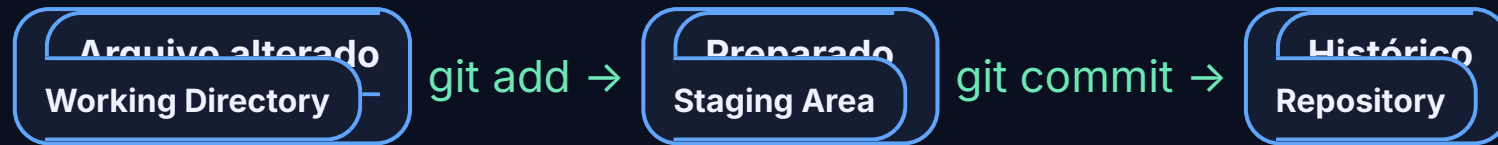
```
git status
```

Mostra a situação do repositório: arquivos novos, modificados, preparados para commit e branch atual.

Regra de estudo: ficou em dúvida? Rode `git status`.

07 · FLUXO LOCAL

Working → Staging → Commit



O Git permite escolher exatamente quais mudanças entram no próximo “ponto salvo”.

08 · STAGE

Prepare arquivos para o commit

Terminal / Git Bash

```
git add README.md  
# ou todos os arquivos da pasta  
git add .  
  
git status
```

`git add` não envia nada para a internet. Ele apenas coloca mudanças na área de preparação.

09 · COMMIT

Crie um ponto no histórico

Terminal / Git Bash

```
git commit -m "docs: adiciona README inicial"
```

Um commit registra as alterações preparadas. Use mensagens curtas que expliquem **o que mudou**.

Exemplos: feat: adiciona cadastro · fix: corrige validação · docs: atualiza README

10 · HISTÓRICO

Veja seus commits

Terminal / Git Bash

```
git log --oneline
```

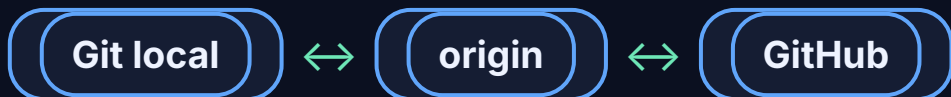
Cada commit recebe um identificador. O log permite acompanhar a evolução do projeto.

```
7f31abc docs: adiciona README inicial
```

11 · GITHUB

Crie o repositório remoto

No GitHub, crie um repositório vazio, por exemplo projeto-git. Para este exercício com código local já existente, evite inicializar o remoto com README, licença ou .gitignore.



12 · REMOTE

Conecte local e GitHub

Terminal / Git Bash

```
git remote add origin https://github.com/SEU-USUARIO/projeto-git.git  
git remote -v
```

origin é apenas o nome/apelido normalmente usado para o repositório remoto principal.

Se aparecer “remote origin already exists”:

```
git remote set-url origin NOVA-URL
```

13 · PRIMEIRO ENVIO

Push para o GitHub

Terminal / Git Bash

```
git push -u origin main
```

push envia commits locais. -u define o vínculo da branch local main com a branch remota; depois, normalmente, basta git push.

14 · CICLO DIÁRIO

O fluxo que você repetirá

Terminal / Git Bash

```
git status  
git add .  
git commit -m "feat: descreva a alteração"  
git push
```



15 · ATUALIZAÇÃO

Traga mudanças do remoto

Terminal / Git Bash

```
git pull
```

Antes de começar a trabalhar em um projeto compartilhado, atualize sua branch local. O pull busca e integra alterações do remoto.

16 · CLONAR

Quando o projeto já existe no GitHub

Terminal / Git Bash

```
git clone https://github.com/USUARIO/REPOSITARIO.git  
cd REPOSITARIO
```

clone cria uma cópia local completa do repositório e já configura o remoto origin.

17 · BRANCHES

Trabalhe sem mexer diretamente na main

Terminal / Git Bash

```
git switch -c feature/login  
# faça alterações  
git add .  
git commit -m "feat: cria tela de login"  
git push -u origin feature/login
```



18 · IGNORAR ARQUIVOS

.gitignore evita lixo e segredos

```
# exemplo
node_modules/
target/
build/
.env
*.log
.idea/
```

Não versione dependências geradas, builds, configurações locais ou arquivos com segredos. Nunca envie senhas, tokens ou chaves de API.

Comandos que ajudam muito

git diff

Mostra alterações ainda não preparadas.

git branch

Lista branches locais.

git remote -v

Mostra os remotos configurados.

git restore arquivo

Descarta alteração local não preparada.

git restore --staged arquivo

Retira arquivo do stage.

git log --oneline

Mostra histórico compacto.

20 · DESAFIO FINAL

Prática do início ao fim

1. Configure nome e e-mail.
2. Crie projeto-git e um README.
3. Inicialize Git na branch main.
4. Faça add + primeiro commit.
5. Crie repositório vazio no GitHub.
6. Configure origin e confira com remote -v.
7. Faça o primeiro push -u origin main.
8. Crie uma branch, altere o projeto e envie a branch.

RESUMO

Mapa mental

```
CONFIGURAR → git config
CRIAR      → git init
VER        → git status / git diff
PREPARAR   → git add
REGISTRAR  → git commit
CONECTAR   → git remote add origin URL
ENVIAR     → git push
ATUALIZAR  → git pull
COPIAR     → git clone
RAMIFICAR  → git switch -c minha-branch
```

Domine primeiro este fluxo. Depois avance para merge, pull request, conflitos, rebase, tags e colaboração em equipe.